

The Mathematics of Vector Search in RAG

May 7, 2026

The Big Picture: RAG at Billion-Scale

The Goal: Retrieve contextually relevant documents from a corpus of $N > 10^9$ to ground Large Language Models (LLMs).

The Pipeline:

1. **Embedding (Bi-Encoders):** Map text to vectors $x \in \mathbb{R}^D$.
2. **Quantization (PQ):** Compress x to reduce RAM footprint.
3. **Partitioning (IVF):** Voronoi-based space division to avoid $O(N)$ scans.
4. **Graph Routing (HNSW):** Sub-logarithmic navigation between partitions.

1. Vector Spaces and Distances

How do we mathematically define if two sentences are "similar"? We measure the distance between their coordinates.

Technical Deep Dive: Given query q and document x in $\mathbb{R}^D$ (typically $D \in [384, 1536]$):

- **Euclidean Distance (L_2):** $d(q, x) = \sqrt{\sum_{i=1}^D (q_i - x_i)^2}$
- **Inner Product (IP):** $IP(q, x) = \langle q, x \rangle = \sum_{i=1}^D q_i x_i$
- **Cosine Similarity:** $S_C(q, x) = \frac{\langle q, x \rangle}{\|q\|_2 \|x\|_2}$

Note: If vectors are L_2 -normalized (i.e., $\|x\| = 1$), then minimizing L_2 distance is mathematically equivalent to maximizing Inner Product.

The Curse of Dimensionality

In 2D space, points can be close or far. In 768D space, everything is practically the same distance apart, making exact search useless.

Technical Deep Dive: By the law of large numbers, the pairwise distances between random vectors in high-dimensional space concentrate around the mean.

$$\lim_{D \rightarrow \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} = 0$$

Conclusion: Exact k -NN ($O(N \cdot D)$ complexity) fails at scale. We must use **ANN** (**Approximate Nearest Neighbors**) to trade a small loss in Recall for a logarithmic speedup.

2. Product Quantization (PQ) and ADC

Product Quantization (PQ): Compression

Slicing a long vector into chunks and replacing each chunk with an ID from a dictionary.

Technical Deep Dive:

1. **Decomposition:** Split $x \in \mathbb{R}^D$ into m orthogonal subspaces: $\mathbb{R}^D = \mathbb{R}^{D/m} \times \dots \times \mathbb{R}^{D/m}$.
2. **K-Means Training:** Learn a codebook $\mathcal{C}_j$ of size k^* (usually 256) for each subspace j .
3. **Encoding:** Map sub-vector u_j to the nearest centroid: $q_j(u_j) = \arg \min_{c \in \mathcal{C}_j} \|u_j - c\|^2$.

Memory Footprint: A standard vector is $D \times 4$ bytes (float32). With PQ ($m = 8$, $k^* = 256$), we store 8 indices of 1 byte each. Compression ratio: $\frac{768 \times 4}{8} = 384\times$.

Quantization Error (MSE): PQ minimizes the distortion $\mathcal{E}$ assuming subspaces are statistically independent:

$$\mathcal{E} = \sum_{j=1}^m \mathbb{E} [\|u_j - q_j(u_j)\|^2]$$

Trade-off: Increasing m reduces $\mathcal{E}$ (better accuracy) but increases storage size.

ADC: Asymmetric Distance Computation

We don't unpack the data to search. We create a cheat sheet (Lookup Table) for the query, and just add numbers up.

To find the squared L_2 distance between an uncompressed query y and a PQ-compressed vector x :

$$\hat{d}(y, x)^2 = \sum_{j=1}^m \|u_j(y) - c_{j, i_j(x)}\|^2$$

Complexity:

1. Build LUT: $O(m \times k^* \times \frac{D}{m}) = O(D \cdot k^*)$ operations (done once).
2. Search: $O(m)$ additions per vector in the database (zero multiplications).

3. Inverted File Index (IVF)

IVF: Voronoi Partitioning

Grouping vectors into buckets so we only search 1% of the database.

Technical Deep Dive: We run K-Means on the entire dataset to find K coarse centroids (parameter `nlist`). This divides the space into Voronoi cells.

Search Parameters:

- `nlist`: Number of clusters (e.g., $4\sqrt{N}$).
- `nprobe`: Number of clusters to visit during search.

Complexity: Search space is reduced from N to approximately $N \cdot \frac{nprobe}{nlist}$.

IVF-PQ: The Residual Trick

Instead of compressing the exact coordinates, we compress the distance from the bucket's center. Smaller numbers = better compression.

Technical Deep Dive: In IVF-PQ, we do not quantize x . We quantize the **residual** vector $r(x)$:

$$r(x) = x - c_{\text{best}}$$

Where c_{best} is the coarse centroid.

Mathematical Justification: The energy (variance) of the residual $r(x)$ is strictly bounded by the Voronoi cell radius. Lower variance in the input data leads to a strictly lower Quantization Error $\mathcal{E}$ during the PQ phase.

4. Graph Algorithms: HNSW and DiskANN

HNSW: Hierarchical Navigable Small World

A layered map of highways and local streets. You jump across the map on the highway, then drop down to local streets to find the exact house.

Technical Deep Dive: HNSW is based on Skip-Lists and proximity graphs.

- **Layer Assignment:** Maximum layer $l = \lfloor -\ln(U(0, 1)) \cdot m_L \rfloor$. This ensures a power-law degree distribution.
- **Parameters:**
 - M : Maximum number of edges per node (memory overhead).
 - $efSearch$: Size of the dynamic candidate list during query time (controls latency vs. recall).
- **Search Complexity:** $O(\log N)$ average case.

DiskANN: Optimizing for SSDs (Vamana Graph)

HNSW requires too much RAM for 1 Billion vectors. DiskANN stores the heavy graph on a hard drive (SSD) and organizes it so the computer reads it efficiently.

SSDs read data in 4KB pages. DiskANN's **Vamana** algorithm controls the graph's out-degree to ensure node neighborhoods fit into single SSD page fetches.

The Pruning Rule (α): When building edges from u , a candidate neighbor v is pruned if another neighbor p already satisfies:

$$\alpha \cdot d(p, v) \leq d(u, v) \quad (\text{where } \alpha \geq 1)$$

This forces long-range "expressway" connections, breaking local clusters and minimizing the number of SSD I/O hops required to reach the target.

Summary

Summary: The Billion-Scale Vector Stack

Algorithm	Mathematical Role	Engineering Benefit
Embeddings	$\phi : \text{Text} \rightarrow \mathbb{R}^D$	Captures semantic topology.
PQ	Subspace K-Means ($x \approx \sum q_i$)	$O(1)$ memory reduction (e.g., 64x).
ADC	Look-Up Table L_2 estimation	Replaces $O(D)$ FLOPS with $O(m)$ additions.
IVF	Voronoi Partitioning	Limits search space to $\frac{n_{\text{probe}}}{n_{\text{list}}}$.
HNSW	Greedy Routing in $O(\log N)$	Sub-millisecond cluster matching.
DiskANN	I/O Bounded Graph Pruning (α)	Bypasses RAM limits using NVMe SSDs.